

17_Mixture-of-R recursions 数学过程详解

动态递归路由、负载均衡与 KV 缓存复杂度

本文只处理真正需要推导的部分

本文推导 expert-choice/token-choice 路由、平均递归深度、KV 内存比、注意力 FLOPs 比、硬门梯度与因果泄漏。全文按“公式从哪里来–每个符号是什么–中间步骤为何成立–维度和复杂度如何核对–论文结论依赖哪些假设”的顺序展开；不复述实验表格，也不使用与论文无关的通用背景凑页数。

目录

1	递归参数共享的基本计算图	3
2	Expert-choice 路由	3
2.1	分位数与 top- k 的等价	3
3	层级过滤与有效深度分布	3
4	Token-choice 路由	4
5	负载均衡损失	4
6	等计算预算下的容量序列	4
7	Recursion-wise KV 缓存内存比例	5
8	Recursive KV sharing 的内存比例	5
9	注意力 FLOPs 比例	5
10	路由门的可导性	6
11	参数共享与梯度累积	6
12	因果泄漏问题	6
13	结论边界	6

14 最终公式路线图	7
15 共享递归块与有效深度	8
16 Token-choice 路由的概率模型	8
17 Expert-choice 与 Token-choice 的容量差异	8
18 层级过滤与有效深度分布	9
19 硬路由的 Straight-Through 梯度	9
20 负载均衡损失的梯度方向	9
21 参数共享下的梯度累积与时间 Jacobian	10
22 KV 缓存的三种内存公式	10
23 一个四 token 的路由例子	10
24 因果泄漏为何会发生	11

1 递归参数共享的基本计算图

设共享递归块为 $f_{\Phi'}$, 第 r 次递归的 token 表示为 $H_t^r \in \mathbb{R}^D$ 。固定深度递归模型执行

$$H_t^{r+1} = f_{\Phi'}(H_t^r), \quad r = 1, \dots, N_r. \quad (1.1)$$

参数量只存一份 Φ' , 但 FLOPs 仍随递归次数近似线性增长。Mixture-of-RecurSIONs 的核心是为每个 token 学习不同的有效递归深度 d_t 。

2 Expert-choice 路由

第 r 个递归专家根据当前隐藏状态计算标量分数

$$g_t^r = \mathcal{G}((\theta_r)^\top H_t^r), \quad (2.1)$$

$\mathcal{G}$ 可为 sigmoid 或 tanh。令 $P_\beta(G_r)$ 为本批次所有分数的 β 分位数阈值, 则更新

$$H_t^{r+1} = \begin{cases} g_t^r f_{\Phi'}(H_t^r) + H_t^r, & g_t^r > P_\beta(G_r), \\ H_t^r, & \text{otherwise.} \end{cases} \quad (2.2)$$

分位数选择保证每层近似固定容量。若保留比例为 c_r , 则每层处理 token 数约为 $c_r N_{\text{ctx}}$ 。

2.1 分位数与 top- k 的等价

若共有 N 个分数且希望选 k 个, 取

$$\beta = 1 - \frac{k}{N} \quad (2.3)$$

的经验分位数。没有并列值时, $g_t > P_\beta$ 与 top- k 等价; 有并列值时需额外 tie-breaking 才能保证严格容量。

3 层级过滤与有效深度分布

论文要求只有第 r 步被选中的 token 才能进入第 $r+1$ 步。令二值门

$$m_t^r = \mathbf{1}[g_t^r > P_\beta(G_r)]. \quad (3.1)$$

层级约束为

$$m_t^{r+1} \leq m_t^r. \quad (3.2)$$

有效递归深度可写为

$$d_t = \sum_{r=1}^{N_r} m_t^r. \quad (3.3)$$

因此 d_t 是非负整数, 并且越难的 token 可被路由到更深层。

4 Token-choice 路由

Token-choice 在第一次递归前一次性决定深度。路由器输出

$$g_t = \text{softmax}(\Theta^\top H_t^1) \in \mathbb{R}^{N_r}, \quad (4.1)$$

并选择

$$i_t = \arg \max_{j \in \{1, \dots, N_r\}} g_{t,j}. \quad (4.2)$$

i_t 表示 token 需要执行的递归次数。递归更新可概括为

$$H_t^{r+1} = \begin{cases} g_{t,r} f_{\Phi'}(H_t^r) + H_t^1, & r = i_t, \\ g_{t,r} f_{\Phi'}(H_t^r), & r < i_t. \end{cases} \quad (4.3)$$

该设计没有每层 top- k 使用未来批次统计导致的因果泄漏，但不同专家可能负载失衡。

5 负载均衡损失

若批次中分配到专家 j 的比例为

$$f_j = \frac{1}{N} \sum_{t=1}^N \mathbf{1}[i_t = j], \quad (5.1)$$

平均路由概率为

$$p_j = \frac{1}{N} \sum_{t=1}^N g_{t,j}, \quad (5.2)$$

常见 MoE 均衡项为

$$\mathcal{L}_{\text{bal}} = N_r \sum_{j=1}^{N_r} f_j p_j. \quad (5.3)$$

当路由均匀时 $f_j = p_j = 1/N_r$ ，损失为 1。若少数专家同时拥有高分配比例和高概率，乘积增大，梯度推动分布展开。

6 等计算预算下的容量序列

若 token-choice 完美均衡，每个 token 的最终深度 $i \in \{1, \dots, N_r\}$ 等概率。第 j 次递归会处理所有 $i \geq j$ 的 token，因此处理比例

$$c_j = \mathbb{P}(i \geq j) = \frac{N_r - j + 1}{N_r}. \quad (6.1)$$

总平均递归次数

$$\mathbb{E}[d] = \sum_{j=1}^{N_r} c_j \quad (6.2)$$

$$= \frac{1}{N_r} \sum_{s=1}^{N_r} s \quad (6.3)$$

$$= \left\lfloor \frac{N_r + 1}{2} \right\rfloor. \quad (6.4)$$

Expert-choice 为公平比较使用同样的容量序列

$$1, \frac{N_r - 1}{N_r}, \dots, \frac{1}{N_r}. \quad (6.5)$$

7 Recursion-wise KV 缓存内存比例

标准固定递归模型为每个深度保存完整 KV，归一化内存为 $N_r N_{\text{ctx}}$ 。若第 j 层只缓存比例 c_j ，总缓存 token 数

$$N_{\text{ctx}} \sum_{j=1}^{N_r} c_j = N_{\text{ctx}} \frac{N_r + 1}{2}. \quad (7.1)$$

相对比例

$$R_{\text{KV}}^{\text{cache}} = \frac{N_r + 1}{2N_r}. \quad (7.2)$$

当 $N_r \rightarrow \infty$ 时趋近 $1/2$ ，说明该方案最多约节省一半递归块 KV，而不是随递归数无限缩小。

8 Recursive KV sharing 的内存比例

若只在第一次递归缓存完整 KV，后续全部共享，则总缓存为 N_{ctx} ，相对固定深度模型

$$R_{\text{KV}}^{\text{share}} = \frac{1}{N_r}. \quad (8.1)$$

内存节省更大，但每一深度的 query 仍需读取同一份完整 KV，因此 IO 比例不一定同样降低。

9 注意力 FLOPs 比例

全注意力单层主要成本与

$$QK^\top : O(N_q N_k D) \quad (9.1)$$

成正比。

Recursion-wise caching 中，query 和 key 都只含 k 个激活 token：

$$R_{\text{attn}}^{\text{cache}} = \frac{k^2}{N_{\text{ctx}}^2} = \left(\frac{k}{N_{\text{ctx}}} \right)^2. \quad (9.2)$$

Recursive sharing 中 query 只有 k ，key 仍为全长 N_{ctx} ：

$$R_{\text{attn}}^{\text{share}} = \frac{k N_{\text{ctx}}}{N_{\text{ctx}}^2} = \frac{k}{N_{\text{ctx}}}. \quad (9.3)$$

这解释了为什么共享方案内存最省，但注意力 FLOPs 降幅不如局部缓存平方级。

10 路由门的可导性

硬 top- k 与 $\arg \max$ 不可导。训练时常把路径选择视作停止梯度，并通过被选 token 的连续门值 g_t^r 传梯度：

$$H_t^{r+1} = m_t^r g_t^r f(H_t^r) + (1 - m_t^r) H_t^r. \quad (10.1)$$

对路由分数

$$\frac{\partial H_t^{r+1}}{\partial g_t^r} = m_t^r f(H_t^r). \quad (10.2)$$

未被选择的 token 得不到主任务梯度，因此需要辅助路由器、均衡损失或软路由预热避免门控早期塌缩。

11 参数共享与梯度累积

共享块 Φ' 在多个递归步被调用，总梯度为

$$\nabla_{\Phi'} \mathcal{L} = \sum_{r=1}^{N_r} \sum_{t:m_t^r=1} \frac{\partial \mathcal{L}}{\partial H_t^{r+1}} \frac{\partial f_{\Phi'}(H_t^r)}{\partial \Phi'} g_t^r. \quad (11.1)$$

因此同一参数同时接收不同深度与不同 token 难度的信号。共享减少参数，却可能造成递归步间梯度冲突；入口/出口层不共享可缓解所有层完全同质化。

12 因果泄漏问题

Expert-choice 在训练时对整段序列分数排序。对位置 t 的阈值

$$P_\beta(G_r) \quad (12.1)$$

可能依赖未来位置 $s > t$ 的分数，因此当前 token 是否继续计算间接看到了未来统计，违反自回归因果性。推理时未来 token 不存在，路由分布可能偏移。辅助路由器的目标是用仅依赖当前/过去信息的函数近似训练时 top- k 决策。

13 结论边界

动态深度的三个成本不能混为一谈

参数量由共享决定，FLOPs 由平均激活递归次数决定，KV 内存/IO 又由缓存策略决定。共享参数并不自动减少 FLOPs；减少 KV 内存也不必然减少 KV 读取；top- k 静态容量保证负载，却可能引入因果泄漏。

14 最终公式路线图

阅读完成后应掌握

本文的数学主线已经被压缩为：先明确随机变量或张量的定义，再由概率分解、微分方程、优化目标或矩阵运算得到论文核心公式；最后用维度、极限情形、参数量和复杂度进行反向核验。遇到论文中的简写式，应先恢复被省略的条件变量、求和指标与归一化常数，再讨论其算法含义。

15 共享递归块与有效深度

设同一个 Transformer 块 F_θ 重复应用 R 次:

$$h^{(r+1)} = F_\theta(h^{(r)}), \quad r = 0, \dots, R-1. \quad (15.1)$$

若每个 token i 有停止深度 d_i , 则其最终表示为

$$y_i = h_i^{(d_i)}. \quad (15.2)$$

整个 batch 的平均有效深度

$$\bar{d} = \frac{1}{BT} \sum_{b,i} d_{bi} \quad (15.3)$$

直接决定主要计算量。参数量只包含一份 θ , 而计算量仍与 $\bar{d}$ 成正比。

16 Token-choice 路由的概率模型

令路由器在第 r 次递归给 token i 输出继续概率

$$p_{ir} = \sigma(s_{ir}). \quad (16.1)$$

若每次独立做 Bernoulli 决策, token 至少执行到第 r 次的生存概率为

$$S_{ir} = \prod_{k=1}^{r-1} p_{ik}. \quad (16.2)$$

停止于第 r 次的概率

$$\Pr(d_i = r) = S_{ir}(1 - p_{ir}), \quad (16.3)$$

最后一层强制停止时需把剩余概率并入 $r = R$ 。期望深度可用尾和公式:

$$\mathbb{E}[d_i] = \sum_{r=1}^R \Pr(d_i \geq r) = \sum_{r=1}^R S_{ir}. \quad (16.4)$$

这给出一个可微的计算预算正则

$$\mathcal{L}_{\text{compute}} = \lambda \frac{1}{BT} \sum_{b,i,r} S_{bir}. \quad (16.5)$$

17 Expert-choice 与 Token-choice 的容量差异

Token-choice 中每个 token 自己选择是否继续, 某轮被选 token 数

$$N_r = \sum_i \mathbf{1}(s_{ir} > \tau_r) \quad (17.1)$$

是随机的, 可能导致负载不均。Expert-choice 则由第 r 个递归“专家”选 top- C_r 个 token:

$$\mathcal{I}_r = \text{TopK}(s_{1:T,r}, C_r). \quad (17.2)$$

于是每轮计算量严格固定为 C_r 。但 token 的选择不再独立, 某 token 是否被选取取决于其他 token 的分数排序。

18 层级过滤与有效深度分布

若要求“进入第 $r + 1$ 轮必须先进入第 r 轮”，掩码满足

$$m_{i,r+1} \leq m_{ir}. \quad (18.1)$$

可用累乘构造

$$m_{ir} = \prod_{k=1}^r g_{ik}, \quad g_{ik} \in \{0, 1\}. \quad (18.2)$$

于是深度分布天然形成嵌套集合

$$\mathcal{I}_{r+1} \subseteq \mathcal{I}_r. \quad (18.3)$$

这防止 token “跳过中间递归后又重新出现”，保持计算图因果一致。

19 硬路由的 Straight-Through 梯度

前向硬门

$$m = \mathbf{1}(p > \tau) \quad (19.1)$$

对分数的真实导数几乎处处为零。STE 可写为

$$\tilde{m} = \text{sg}(m - p) + p. \quad (19.2)$$

前向数值为 m ，因为 $\text{sg}(m - p)$ 保留值；反向

$$\frac{\partial \tilde{m}}{\partial s} = \frac{\partial p}{\partial s} = p(1 - p). \quad (19.3)$$

因此路由器能收到梯度，但该梯度并非离散 top- k 目标的真实梯度。温度降低会使 p 更接近 0/1，同时令 $p(1 - p)$ 变小，存在探索与梯度消失权衡。

20 负载均衡损失的梯度方向

设第 r 轮理想负载比例为 q_r ，实际软负载

$$\hat{q}_r = \frac{1}{BT} \sum_i p_{ir}. \quad (20.1)$$

平方负载损失

$$\mathcal{L}_{\text{bal}} = \sum_r (\hat{q}_r - q_r)^2. \quad (20.2)$$

对某个 logit s_{ir} ,

$$\frac{\partial \mathcal{L}_{\text{bal}}}{\partial s_{ir}} = \frac{2}{BT} (\hat{q}_r - q_r) p_{ir} (1 - p_{ir}). \quad (20.3)$$

若该轮负载过高，梯度下降会降低所有相关 logits；若负载过低，则提升它们。

21 参数共享下的梯度累积与时间 Jacobian

总损失对共享参数

$$\nabla_{\theta} \mathcal{L} = \sum_{r=0}^{R-1} \left(\frac{\partial h^{(r+1)}}{\partial \theta} \right)^{\top} \nabla_{h^{(r+1)}} \mathcal{L}. \quad (21.1)$$

状态梯度沿递归展开：

$$\frac{\partial h^{(R)}}{\partial h^{(0)}} = \prod_{r=0}^{R-1} J_r, \quad J_r = \frac{\partial F_{\theta}(h^{(r)})}{\partial h^{(r)}}. \quad (21.2)$$

共享参数不消除 BPTT 式 Jacobian 连乘，因此递归很深时仍可能梯度爆炸或消失。残差形式 $F(h) = h + G(h)$ 把每个因子变为 $I + J_G$ 。

22 KV 缓存的三种内存公式

设序列长度 T ，每层 KV 维度总计 $2d_{kv}$ ，字节数 b 。普通 L 层模型缓存

$$M_{\text{standard}} = 2LTd_{kv}b. \quad (22.1)$$

若共享递归块但每次递归保留独立 KV，

$$M_{\text{recursion}} = 2RTd_{kv}b. \quad (22.2)$$

若 recursion-wise KV sharing，只保留一份，

$$M_{\text{shared}} = 2Td_{kv}b. \quad (22.3)$$

相对普通模型比例为

$$\frac{M_{\text{shared}}}{M_{\text{standard}}} = \frac{1}{L}. \quad (22.4)$$

但共享 KV 意味着不同递归深度无法保存各自的键值表示，表达能力可能下降。

23 一个四 token 的路由例子

四个 token 在三轮的继续概率为

$$P = \begin{pmatrix} 0.9 & 0.8 & 0.5 \\ 0.8 & 0.2 & 0.1 \\ 0.6 & 0.6 & 0.6 \\ 0.3 & 0.2 & 0.1 \end{pmatrix}. \quad (23.1)$$

第一个 token 的期望深度

$$\mathbb{E}[d_1] = 1 + 0.9 + 0.9 \times 0.8 = 2.62. \quad (23.2)$$

第四个 token

$$\mathbb{E}[d_4] = 1 + 0.3 + 0.3 \times 0.2 = 1.36. \quad (23.3)$$

因此路由器把更多计算分配给认为更困难的 token。若使用 top-2 expert-choice, 则每轮恰好两个 token 继续, 计算固定, 但期望深度公式需基于排序事件而不是独立 Bernoulli。

24 因果泄漏为何会发生

在自回归生成中, 第 i 个 token 的路由决策只能依赖 $x_{\leq i}$ 。若为了全序列负载平衡, 用未来 token 分数共同决定当前 token 是否进入递归,

$$m_i = \text{TopK}(s_{1:T})_i, \quad (24.1)$$

则 m_i 依赖 $s_j, j > i$, 从而间接泄漏未来信息。安全做法是逐前缀决策、训练时使用因果掩码, 或只在已经给定的 prompt token 上做全局 expert-choice。